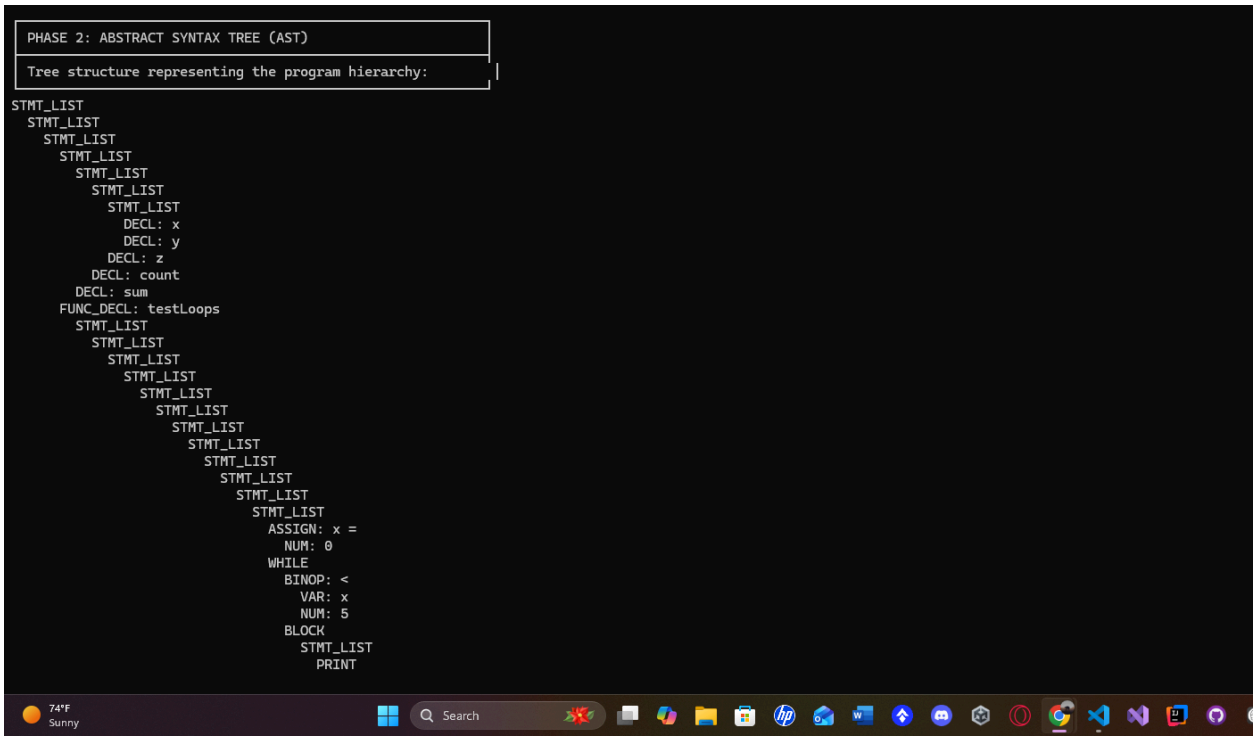
I am most proud of my constant folding optimization that evaluates expressions like 2 + 3 * 4 at compile-time to produce 14 directly, eliminating unnecessary runtime calculations by pattern-matching binary operations with literal operands and replacing the entire subtree with a constant node.

```
99: t39 = t38 * 3
100: t40 = 5 + t39     // Add: store result in t40
101: /* unknown op 18 */
102: PRINT 600          // Output value of 600
103: LABEL L13
104: t42 = 2 * t41
105: t43 = 10 - t42     // Subtract: store result in t43
106: /* unknown op 18 */
107: PRINT 601          // Output value of 601
108: LABEL L14
109: count = 0           // Assign value to count
110: LABEL L15
111: t44 = 2 + 3     // Add: store result in t44
112: /* unknown op 18 */
113: PRINT 700          // Output value of 700
114: t46 = count + 1     // Add: store result in t46
115: count = t46           // Assign value to count
116: /* unknown op 17 */
117: LABEL L16
118: FUNC_END testOrderOfOperations
119: FUNC_BEGIN main
120: RET 0
121: FUNC_END main

PHASE 4: CODE OPTIMIZATION

Applying optimizations:
• Constant folding (evaluate compile-time expressions)
• Copy propagation (replace variables with values)
```

My hierarchical symbol table correctly handles variable shadowing across nested scopes using a hash map at each scope level with parent pointers, enabling O(1) lookups and graceful fallback to outer scopes when variables aren't found locally.



The three-address code intermediate representation serves as an elegant abstraction between AST and assembly, with each instruction containing at most one operator and three operands (like t1 = a + b), which maps cleanly to assembly while remaining human-readable for debugging and optimization.z

```
77: PRINT x           // Output value of x
78: t25 = 20 * 4
79: t26 = t25 * 2
80: y = t26           // Assign value to y
81: PRINT y          // Output value of y
82: t27 = 3 * 2
83: t28 = 5 + t27     // Add: store result in t28
84: t29 = 8 * 4
85: t30 = t28 - t29     // Subtract: store result in t30
86: z = t30           // Assign value to z
87: PRINT z          // Output value of z
88: t31 = 5 + 3     // Add: store result in t31
89: t32 = t31 * 2
90: t33 = 8 * 4
91: t34 = t32 - t33     // Subtract: store result in t34
92: x = t34           // Assign value to x
93: PRINT x          // Output value of x
94: t36 = 3 * t35
95: t37 = 2 + t36     // Add: store result in t37
96: /* unknown op 18 */
97: PRINT 500          // Output value of 500
98: LABEL L12
99: t39 = t38 * 3
100: t40 = 5 + t39     // Add: store result in t40
101: /* unknown op 18 */
102: PRINT 600          // Output value of 600
103: LABEL L13
104: t42 = 2 * t41
105: t43 = 10 - t42     // Subtract: store result in t43
106: /* unknown op 18 */
107: PRINT 601          // Output value of 601
108: LABEL L14
109: count = 0           // Assign value to count
110: LABEL L15
111: t44 = 2 + 3     // Add: store result in t44
112: /* unknown op 18 */
113: PRINT 700          // Output value of 700
114: t46 = count + 1     // Add: store result in t46
115: count = t46           // Assign value to count
116: /* unknown op 17 */
```